

実験ログ: 布とやわらかい物 (Vellum)

2026-09-24 | 収録 11 件

Houdini 研究部で、hython で実際に測った実験の記録。数字はどれも測った値。

収録している実験

- Vellum クロス — 布で保存されるべき量は何か
- 布は破れなかった — Vellum の breaking が効かない条件を潰す
- Vellum の breaking は動く — ただし布の伸びには効かない
- 狙った線で破れる布 — 先に切って、あとから貼る
- 布を物にぶつけて破る — 破れ始めは当たった場所、そのあとは全体へ
- 毛を揺らす — 曲がりにくさは「垂れる距離」ではなく「形」に効いていた
- 毛に風を当てる — 実験029と同じ「二次の抵抗」だった
- 旗を風になびかせる — 風速 2 まではほぼ垂れたまま。持ち上がり方は「抵抗 × 風速²」でそう

- vellumconstraints の Cloth

は網を三角形にしてから、辺ごとに伸びの拘束、内側の辺ごとに曲げの拘束を作る

- Vellum の布は既定の硬さでも Substeps 1 で 2% 伸びる — Substeps 1 では 10^4 と 10^{10} の差が出ない
- Vellum の布の伸びは、Substeps より Constraint Iterations を上げる方が安く減る — 400 回で 0.2%、1600 回で 0.002%

Vellum クロス — 布で保存されるべき量は何か

2026-09-24 | Houdini 21.0.700 | /obj/vellum_test | ノード 13 / 接続 19

何を作ったか

剛体では体積が保たれた（実験014）。布は面なので体積では測れない。代わりに「辺の長さの合計」を見る。布は伸びにくい素材なので、たわんでも吊るしても、辺の長さの合計はほぼ変わらないはず。

さらに、伸びにくさ（stretchstiffness）を下げれば保存が崩れるはず。そう考えて試したところ、まったく変化しなかった。原因を追ったら、このパラメータの本当の仕組みが分かった。

ノードの接続



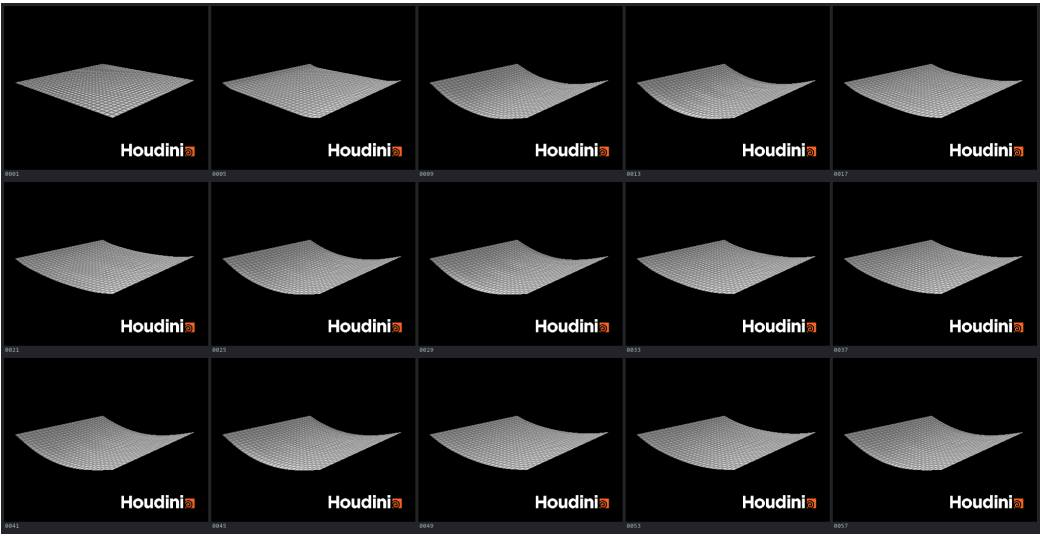
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
cloth	grid	cols = 30 rows = 30 size_x = 4.0 size_y = 4.0 ty = 3.0
collider	sphere	rad_x = 1.2 rad_y = 1.2 rad_z = 1.2 ty = 1.0 type = 1
make_pins	attribwrangle	snippet = if (abs(@P.x) > 1.9) { @group_pins = 1; }
con_drape_stiff	vellumconstraints	constrainttype = 3
solve_drape_stiff	vellumsolver	useground = 1 windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618
con_drape_soft	vellumconstraints	constrainttype = 3 stretchstiffness = 0.02
solve_drape_soft	vellumsolver	useground = 1 windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618
con_hang_stiff	vellumconstraints	constrainttype = 3
pin_hang_stiff	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve_hang_stiff	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618

ノード	タイプ	デフォルトから変更したパラメータ
con_hang_soft	vellumconstraints	constrainttype = 3 stretchstiffness = 0.02
pin_hang_soft	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve_hang_soft	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618

両端をピンで留めて吊るす（15コマ）

4×4 の布（900点 / 841面）の両端をピンで固定し、自重でたわませた。ピンで留めると布に張力がかかるので、伸びにくさの違いが出やすくなる。



15コマ。両端が固定されたまま中央が垂れ下がる

条件ごとの伸び率

「伸び率」は辺の長さの合計を、たわむ前の状態（464.00000）で割った値。1.0 なら一切伸びていない。

条件	最大の伸び率	伸びた割合
たわむ・かたい（stiffness 1.0）	1.00272	0.272%
たわむ・やわらかい（stiffness 0.02）	1.00257	0.257%
吊るす・かたい（stiffness 1.0）	1.00842	0.842%
吊るす・やわらかい（stiffness 0.02）	1.00842	0.842%

かたさを大きく振る（吊るした布、最終フレーム）

stiffness を 50倍変えても結果が同じだったので、指数のパラメータを含めて広く振った。値と指数を入れ替えたペアの結果が完全に一致したので、実効的なかたさは「値 × 10の指数乗」だと確定した。

stretchstiffness	stretchstiffnessexp	実効的なかたさ	最後の伸び率	最下点
1.0	6	10 ⁶	1.00843	2.6042

stretchstiffness	stretchstiffnessexp	実効的なかたさ	最後の伸び率	最下点
1000000.0	0	10^6	1.00843	2.6042
1.0	0	10^0	2.74029	-3.9926
1.0	-6	10^{-6}	3.53114	-5.7265
0.000001	0	10^{-6}	3.53114	-5.7265

わかったこと

- ・点数は全フレームで 900 のまま変わらない。布は形が変わるだけで、点が増減することはない。剛体の体積保存にあたるのが、布ではこの「構造が変わらないこと」。

- ・たわませただけでは、辺の長さの合計はほとんど変わらなかった（最大 0.272% の伸び）。布が自分の重さで垂れるだけでは、張力がほとんどかからないため。

- ・両端をピンで留めて吊ると 0.842% まで増えた。たわむ場合の3.1倍。布を引っ張る力がかかっている分だけ伸びている。指標が張力を捉えられていることが確認できた。

- ・ところが、かたさを 1.0 から 0.02 に下げても結果がまったく変わらなかった。50分の1にしたのに伸び率は 1.00842 で小数点以下5桁まで同じ。

- ・原因は stretchstiffnessexp というパラメータで、既定値が 10 だった。実効的なかたさは「値 × 10の指数乗」なので、既定では 1.0×10^{10} という極端に硬い状態。ここから 0.02 にしても 2×10^8 で、やはり伸びない。

- ・これを確かめるために値と指数を広く振った。(値 1.0, 指数 6) と (値 1000000, 指数 0) の結果が完全に一致し、(値 1.0, 指数 -6) と (値 0.000001, 指数 0) も完全に一致した。掛け算であることが実測で確定した。

- ・指数を 0 まで下げると伸び率 2.74029 (174%の伸び) 、-6 まで下げると 3.53114 (253%の伸び) になり、布というよりゴムのような挙動になる。最下点も 2.60 から -5.73 まで下がる。

- ・実用上の意味は大きい。画面に見えている数字 (1.0) だけを触っても、実際にはほとんど変わらない。効くのは指数のほう。布が硬すぎる・柔らかすぎると感じたなら、値ではなく指数を疑う。

- ・ピンの指定でも1つつまずいた。group type の既定はプリミティブなので、点のグループを渡すなら先に points へ切り替える必要がある。さらに group を空のままにすると「全部が対象」と解釈され、布全体がピン留めされて一切動かなくなる。最下点が全フレーム 3.0000 のまま動かないという分かりやすい症状が出た。

- ・計算は速く、1フレームあたり 0.3~0.8秒。900点の布ならシミュレーションでも実験のテンポは保てる。

次にやること

- ・ bendstiffness にも同じ指数があるか確かめる。あるなら曲がりにくさも同じ罫がある

- ・布と球の衝突で、厚み (thickness) がどう効くかを測る

- ・Pyro (煙) へ進む。ボリウムなので、保存されるべき量がまた変わる

布は破れなかった — Vellum の breaking が効かない条件を潰す

2026-09-24 | Houdini 21.0.700 | /obj/tear | ノード 6 / 接続 7

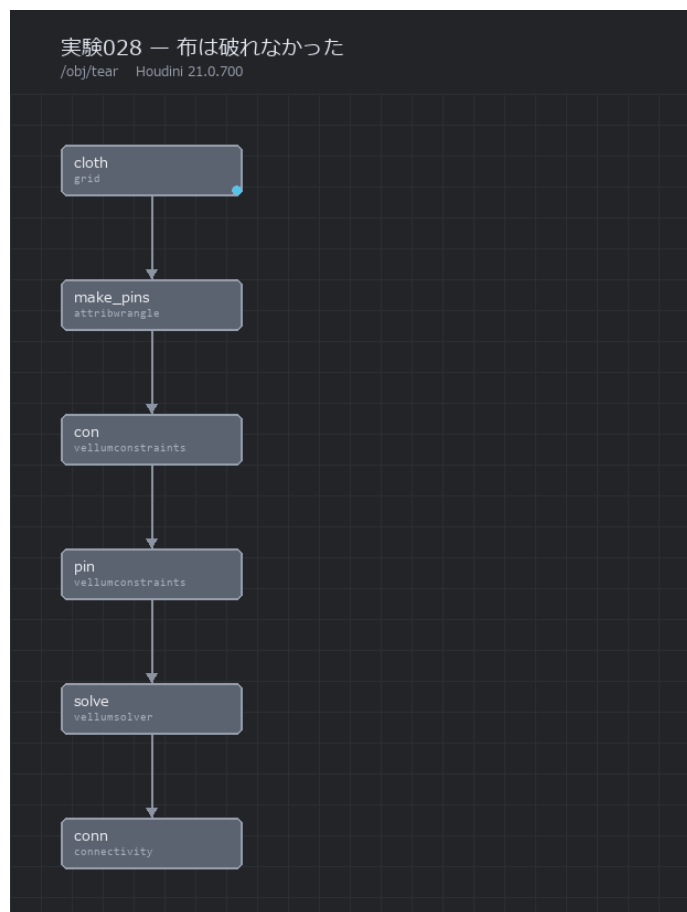
何を作ったか

実験015で布を垂らした。今回はそれを破るつもりだった。vellumconstraints には dobreaking (既定でオフ) と breakthreshold (既定 0.1) がある。しきい値を下げれば破れやすくなるはずである。

結果を先に書く。1つも破れなかった。しきい値を 1.0 から 0.000001 まで下げても、破れ方の種類を3つとも試しても、布を100倍重くして応力を26,552まで上げてても、切れた拘束はゼロのままだった。しきい値の 2.7×10 の10乗倍の応力がかかっても、である。

「効かない」と書くには、効くはずの条件を潰しきる必要がある。5つ潰した。しきい値・破れ方の種類・力の強さ・拘束の型・ソルバの設定。それでも破れない。原因はまだ特定できていないので、断定はしない。確かめたことと、まだ確かめていないことを分けて残す。

ノードの接続

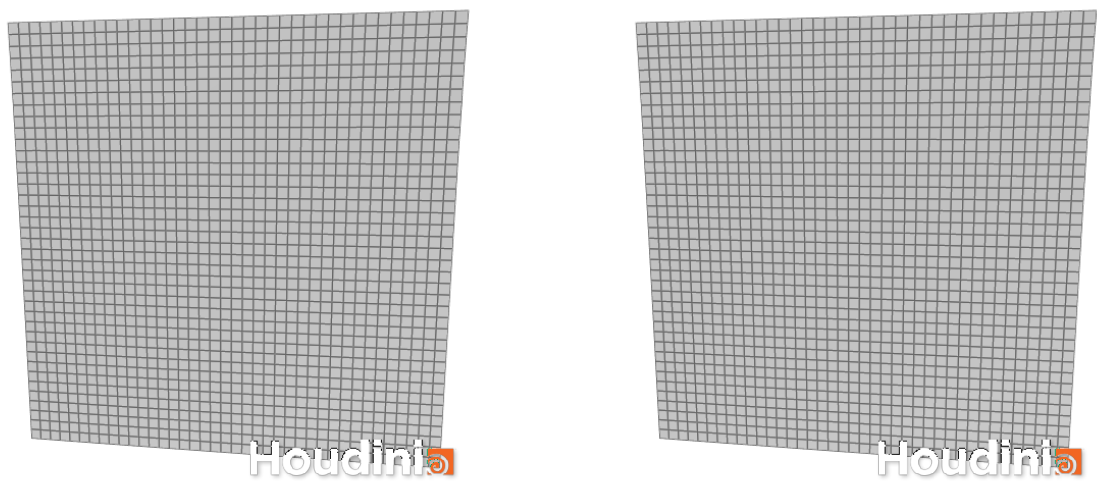


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
cloth	grid	cols = 40 orient = 0 rows = 40 sizex = 4.0 sizey = 4.0 ty = 2.2
make_pins	attribwrangle	snippet = if (@P.y > 4.1) { @group_pins = 1; }
con	vellumconstraints	constrainttype = 3 dobreaking = 1
pin	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618
conn	connectivity	connecttype = 1

組んだもの

4×4の布を40×40に分割し、上の辺1列だけをピンで留めて吊るした。布の全重量が上端の拘束にかかる。48フレーム回す。



breaking を切った状態。F48。

しきい値
0.02。同じカメラ・同じフレーム。見分けがつかない。

項目	値
布	4×4 / 40×40分割 / 1,600点 / 1,521面
留めたところ	上の辺1列だけ
作られた拘束	9,126本 (distance 4,641 + bend 4,485)
拘束の型 / 破れ方	distance / stretchstress
ソルバのモード	full (Minimal ではない)
中の DOP の判定頻度	perframe (Never ではない)

1. しきい値を下げていく

破れ方の種類は既定の stretchstress。48フレーム目の状態を測った。

しきい値	切れた拘束	拘束	点	塊	一番長い辺	最下点
なし（基準）	0	9,126	1,600	1	0.10506	0.1594
1.0	0	9,126	1,600	1	0.10506	0.1594
0.5	0	9,126	1,600	1	0.10506	0.1594
0.2	0	9,126	1,600	1	0.10506	0.1594
0.1（既定値）	0	9,126	1,600	1	0.10506	0.1594
0.05	0	9,126	1,600	1	0.10506	0.1594
0.02	0	9,126	1,600	1	0.10506	0.1594

2. 破れ方の種類を変える（しきい値 0.000001）

breakypestretch の3種類すべて。拘束の属性が正しく書き換わっていることも確認済み。

破れ方	切れた拘束	応力の最大
stretchstress（応力）	0 / 9,126	22.603
stretchdistance（伸びた距離）	0 / 9,126	22.603
stretchratio（伸び率）	0 / 9,126	22.603

3. 力を強くする（しきい値 0.000001）

布を柔らかくし、さらに重さを100倍にした。応力はしきい値の 2.7×10 の10乗倍に達している。

stretchstiffness	重さ	切れた拘束	応力の最大	一番長い辺
1.0	1	0	22.603	0.10506
0.01	1	0	22.605	0.10506
0.001	1	0	22.616	0.10506
0.001	100	0	26,551.953	0.10672

わかったこと

- ・ どの条件でも1つも切れなかった。しきい値7段階・破れ方3種類・力4段階、合わせて14通り。切れた拘束はすべて 0。
- ・ 結果は完全に同一だった。breaking を切った状態と、しきい値 0.02 の状態で、一番長い辺（0.10506）も最下点（0.1594）も小数点以下まで一致する。実験021の shredding と同じ形の「効いていない」である。
- ・ 応力はしきい値をはるかに超えていた。重さ100倍のときの応力の最大は 26,551.953。しきい値は 0.000001。その差は 2.7×10 の10乗倍。「力が足りない」では説明できない。
- ・ 拘束の型は対象だった。cloth が作る拘束のうち 4,641本は型が distance で、breaktype に stretchstress が書かれている。付属ヘルプは breakypestretch を「distance 拘束の破れ方を決める」と説明しており、条件を満たしている。
- ・ ソルバは判定を切っていなかった。SOP の solvermode は full（Minimal モードなら拘束の増減が非対応だが、そうではない）。SOP 中の DOP ソルバの breakfrequency は perframe で、毎フレーム判定している設定だった。

- ・破れた拘束は消えるのではなく broken グループに入る。ソルバの2番目の出力に、その名前のプリミティブグループが用意されている。今回はここが最後まで空のままだった。つまり「消えたかどうか」ではなく「グループに入ったかどうか」で数えるのが正しい。

- ・付属ヘルプに breakfrequency は載っているが、SOP のパラメータには無い。DOP 版の vellumsolver にはあり、SOP 版には出ていない。SOP の中に埋まっている DOP ノードを開いて読むまで、値が分からなかった。

- ・ここまでが確かめたこと。原因は特定できていない。残っている可能性は次の3つ。(1) SOP レベルの vellumsolver では拘束の破れが扱われない、(2) 別の未知の設定が要る、(3) このHoudiniの版に固有の挙動。断定はしない。

次にやること

- ・DOP レベルで vellum を組み、breakfrequency を自分で指定して同じ測定をする
- ・weld (溶接) や glue (接着) の拘束で breaking を試す。付属ヘルプは breaktypeall を「weld と hair の破れ方」と説明しており、そちらが本来の対象かもしれない
- ・029 として POP で粒に力をかける
- ・実験030に達したら、001~030の振り返り点検を行う

Vellum の breaking は動く — ただし布の伸びには効かない

2026-09-24 | Houdini 21.0.700 | /obj/glue | ノード 8 / 接続 10

何を作ったか

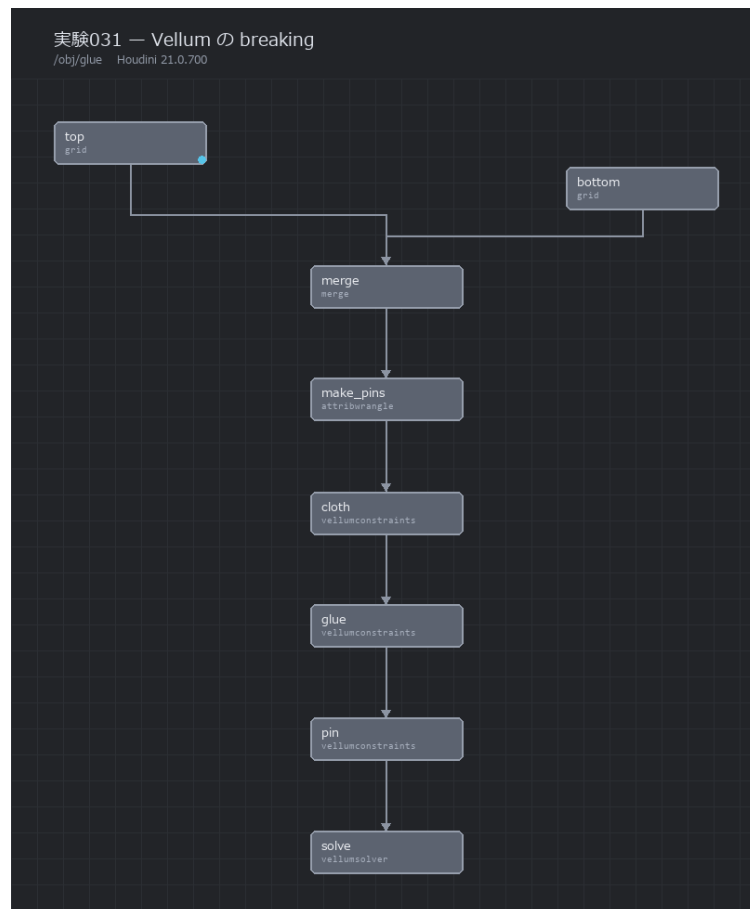
実験028で布を破ろうとして、14通り試して1本も破れなかった。原因は特定できず、3つの可能性を残した。今回それを潰した。

答えが出た。breaking は動く。ただし効く拘束の種類が決まっている。glue が作る stitch 拘束はしきい値どおりに破れ、cloth が作る distance / bend 拘束は同じ場面・同じしきい値でも1本も破れない。

決め手は、1つのシーンの中で両方に breaking を入れたこと。消えたのは stitch の12本だけで、distance 770本と bend 682本は1本も消えていない。solver も、しきい値も、フレーム範囲も、かかる力もすべて同じである。「設定が足りない」のではなく「この種類には効かない」だった。

あわせて、028に書いた説明が逆だったことも分かった。028では「破れた拘束は消えるのではなく broken グループに入る」と書いたが、実際は破れた拘束は取り除かれ、broken グループは空のままである。12本が消えても broken は 0 だった。

ノードの接続

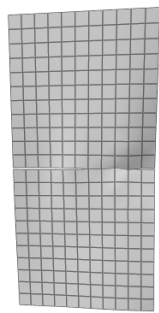


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
top	grid	cols = 12 orient = 0 rows = 12 sizeX = 2.0 sizeY = 2.0 ty = 4.0
bottom	grid	cols = 12 orient = 0 rows = 12 sizeX = 2.0 sizeY = 2.0 ty = 2.02
merge	merge	(変更なし)
make_pins	attribwrangle	snippet = if (@P.y > 4.9) { @group_pins = 1; }
cloth	vellumconstraints	constrainttype = 3
glue	vellumconstraints	breakthreshold = 1e-06 constrainttype = 12 dobreaking = 1
pin	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618

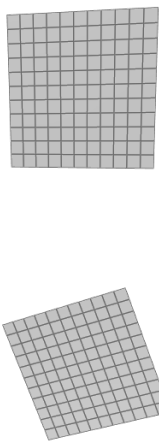
1. 同じ場で、どの種類の拘束が破れるか

上の布を吊るし、その下にもう1枚を glue でくっつけた。下の布の重さは12本の stitch 拘束だけが支える。しきい値はどちらも 0.000001。



Houdini

cloth に breaking。1本も破れず、2枚がつながったまま垂れている。



Houdini

glue に breaking。同じカメラ・同じフレーム。下の布が外れて落ちていく。

設定	distance 770本	bend 682本	stitch 12本	broken グループ	最下点
どちらも breaking なし	0本 消えた	0本	0本	—	0.9608

設定	distance 770本	bend 682本	stitch 12本	broken グループ	最下点
glue だけ breaking	0本	0本	12本 消えた	0	-8.6439
cloth だけ breaking	0本	0本	0本	0	0.9608
両方に breaking	0本	0本	12本 消えた	0	-8.6439

2. しきい値は、ちゃんと効いている

glue 側の breaking のしきい値だけを変えた。拘束の本数がどのフレームで減るかを見る。入力はどれも 1,464本。

しきい値	F3	F10	F40	最下点	起きたこと
なし	1,464	1,464	1,464	0.9608	破れない
0.000001	1,452	1,452	1,452	-8.6439	F3までに12本すべて
1.0	1,452	1,452	1,452	-8.6439	F3までに12本すべて
10.0	1,464	1,456	1,452	-5.6540	少しずつ破れる
100.0	1,464	1,464	1,464	0.9608	破れない
1000.0	1,464	1,464	1,464	0.9608	破れない

3. 028のカーテンで、cloth と distance を比べる

上端だけで吊るした4×4の布。実験028と同じ場面。cloth プリセットを通さない素の distance 拘束でも試した。

拘束の種類	しきい値	拘束の本数（入力 → F40）	最下点
cloth	なし	2,166 → 2,166	0.1929
cloth	0.000001	2,166 → 2,166	0.1929
distance	なし	1,121 → 1,121	0.1931
distance	0.000001	1,121 → 1,121	0.1931
distance	10.0	1,121 → 1,121	0.1931

4. DOP レベルで判定の頻度を指定しても同じ

SOP 版の中の配線をまねて DOP を自分で組むと、SOP には出ていない breakfrequency（破れ判定の頻度）を指定できる。しきい値 0.000001。

breakfrequency	拘束の本数（入力 → F40）	最下点
never（判定しない）	2,166 → 2,166	0.1928
perframe（毎フレーム）	2,166 → 2,166	0.1928
persubstep（毎サブステップ）	2,166 → 2,166	0.1928

わかったこと

- ・ breaking は動いていた。glue が作る stitch

拘束は、しきい値どおりに破れる。12本が消え、下の布が落ちて最下点が 0.9608 から -8.6439 になった。実験028の「1つも破れない」は、拘束の種類によるものだった。

- ・ 決め手は、1つのシーンの中で両方に breaking を入れたこと。消えたのは stitch の12本だけ。distance 770本と bend 682本は1本も消えていない。solver も、しきい値も、フレーム範囲も、かかる力もすべて同じ場面である。条件が同じなら、残る違いは種類しかない。

- ・ しきい値はちゃんと効いている。0.000001 と 1.0 では F3 までに12本すべてが破れ、10.0 では少しずつ破れ（F10で8本、F40で12本）、100.0 以上では1本も破れない。境目は 10 と 100 の間にある。「効かない」ではなく「効く範囲が違う」。

- ・ 028に書いた説明は逆だった。「破れた拘束は消えるのではなく broken グループに入る」と書いたが、実際は取り除かれ、broken グループは空のままである。12本が消えても broken は 0 だった。028では本数も数えていたので結論は変わらないが、理由の説明を訂正する。

- ・ cloth プリセットを通さない素の distance 拘束でも破れない。「cloth プリセットが何か余計なことをしている」という筋は消えた。distance という種類そのものが、この構成では breaking の対象になっていない。

- ・ DOP レベルでも同じだった。SOP 版には出ていない breakfrequency を自分で persubstep（毎サブステップ判定）にしても、拘束は1本も減らない。never との差もゼロ。判定の頻度の問題ではない。

- ・ 実用の結論。布を破りたいなら、cloth の拘束を破ろうとしない。破りたい線に沿ってあらかじめ布を切り分け、glue でくっつけておく。あとはしきい値で、どれだけの力で外れるかを決められる。

- ・ 道具として覚えておくこと2つ。DOP の拘束ジオメトリは dopimport の geodatapath に ConstraintGeometry と書けば取り出せる。もう1つ、normalizestress を切ると stress の値が 7.925 から 0.014 に変わる（566分の1）。同じ「stress」という名前でも、設定によって桁が変わる。

- ・ 028で「断定はしない」と書いて3つの可能性を残したのが効いた。そのうち「SOPレベルでは扱われない」は今回否定され（DOPでも同じ）、残ったのは「別の条件が要る」だった。その条件が拘束の種類だと分かった。見立てを断定せずに残しておく、あとで潰せる。

次にやること

- ・ 切れ目を入れた布を glue でつなぎ、狙った線で破れる布を作る
- ・ 032 以降、まだ触っていない分野に進む
- ・ 実験060で 031~060 の振り返り点検を行う

狙った線で破れる布 — 先に切って、あとから貼る

2026-09-24 | Houdini 21.0.700 | /obj/tear_line | ノード 11 / 接続 14

何を作ったか

実験031で分かったのは、Vellum の breaking は glue が作る stitch 拘束にだけ効き、cloth が作る distance 拘束には効かないということだった。つまり布の伸びを破ることはできないが、貼り合わせを外すことはできる。

ならば順序を逆にすればよい。先に切っておいて、あとから貼る。破れてほしい線で布を2つに切り分け、glue

で貼り直し、しきい値を決めて吊るす。重さが貼り目にかかり、しきい値を超えたところから外れる。

狙いどおりになった。波打つ線で切った布は、その線のかたちのまま裂けた。貼り目の位置は狙った線から平均 0.044、最大でも 0.101

しか離れておらず、これはマス目1つぶん (0.103) より小さい。しきい値 100

以上なら1枚の布として垂れたまま、10 以下なら下半分が外れて落ちる。

ノードの接続

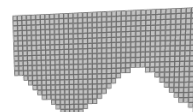
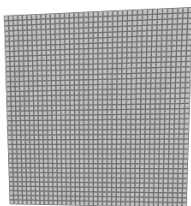


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

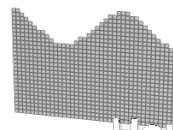
ノード	タイプ	デフォルトから変更したパラメータ
cloth	grid	cols = 40 orient = 0 rows = 40 sizeX = 4.0 sizeY = 4.0 ty = 2.2
mark_cut	attribwrangle	class = 1 snippet = float line = 2.6 + 0.45 * sin(@P.x * 2.2); i@lower = (@P.y < line) ? 1 : 0;
upper	blast	group = @lower==1 grouptype = 4
lower	blast	group = @lower==0 grouptype = 4
two_pieces	merge	(変更なし)
make_pins	attribwrangle	snippet = if (@P.y > 4.1) { @group_pins = 1; }
cloth1	vellumconstraints	constrainttype = 3
glue	vellumconstraints	breakthreshold = 1.0 constrainttype = 12 dobreaking = 1
pin	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618
conn	connectivity	(変更なし)

できたもの

4×4の布を40×40に分割し、上端だけをピンで留めて吊るした。切った線は $y = 2.6 + 0.45 \cdot \sin(2.2x)$ の波。カメラは2枚とも同じ。



Houdini



Houdini

しきい値 1000。1枚の布として垂れる。貼り目は見えない。

しきい値 1.0。切った波の線のまま裂けて、下半分が落ちる。

段取り	やること
1	布を1枚作る (grid)
2	破れてほしい線で面を2組に分ける (attribwrangle でプリミティブに印)
3	2つに切り分けて合流させる (blast ×2 → merge)
4	cloth の拘束を付ける。ピースごとに独立して付く
5	glue で貼り直す。ここに breaking としきい値を入れる
6	上端をピンで留めて解く

1. 貼り目は、狙った線の上にあるか

stitch 拘束は2点を結ぶプリミティブなので、その重心を「その拘束のある場所」とみなして測った。

調べたこと	結果
元の布	1,600点 / 1,521面
切ったあと	1,666点 (+66)
作られた拘束	distance 4,706 / bend 4,420 / stitch 66
stitch の高さの範囲	2.1487 ~ 3.0718
狙った線の範囲	2.15 ~ 3.05 ($y = 2.6 + 0.45 \cdot \sin(2.2x)$)
線からのずれ	平均 0.04405 / 最大 0.10071
マス目の間隔	0.1026 (4 ÷ 39)
検算: 増えた点 = stitch の数	66 = 66

2. しきい値で、外れる／外れないを決められるか

拘束の本数は cloth 9,126本 + stitch 66本 = 9,192本から始まる。外れると stitch が取り除かれて 9,126本になる。

しきい値	F5	F15	F40	吊るした側 (683点)	外れた側 (983点)
0.01	9,126	9,126	9,126	2.140 ~ 4.200	-11.335 ~ -8.448
1.0	9,126	9,126	9,126	2.140 ~ 4.200	-11.055 ~ -8.187
10.0	9,161	9,126	9,126	2.141 ~ 4.200	-8.953 ~ -5.728
100.0	9,192	9,192	9,192	2.116 ~ 4.200	0.155 ~ 3.053
1000.0	9,192	9,192	9,192	2.116 ~ 4.200	0.155 ~ 3.053

3. 切らない布は、やはり破れないまま (031の再確認)

同じ吊るし方で、切らずに cloth の拘束へ直接 breaking を入れた場合。

しきい値	拘束の本数 (入力 → F40)	最下点
0.000001	9,126 → 9,126	0.1667

しきい値	拘束の本数（入力 → F40）	最下点
1.0	9,126 → 9,126	0.1667

わかったこと

- ・狙った線のまま裂けた。貼り目の位置は、狙った線から平均 0.044 ・最大 0.101 しか離れていない。マス目1つぶん (0.1026) より小さい。線をどう引いても、その形のまま破れる。
- ・数が合っている。切ると切り口の点が二重になる。増えた点は 66個で、glue が作った stitch もちょうど 66本。重なった1組につき1本という理屈どおり。ここがずれていたら、貼り忘れか貼りすぎがある。
- ・しきい値で「どれだけの力で外れるか」を決められる。10以下なら外れ、100以上なら外れない。10のときは F5 で 9,161本、F15 で 9,126本と少しずつ外れる。境目は 10 と 100 の間で、実験031の glue のときと同じ。
- ・外れないときは、貼り目がまったく見えない。しきい値 1000 の画は1枚の布そのもので、どこで切ったか分からない。貼っておけば、破れるまでは普通の布として振る舞う。
- ・切らない布は、やはり破れない。同じ吊るし方で cloth の拘束に breaking を入れても、しきい値 0.000001 でも 9,126本のまま。実験031の結論がそのまま再現した。
- ・blast の grouptype でつまずいた。選択肢は 0=推測 / 1=ブレイクポイント / 2=辺 / 3=点 / 4=面。面のつもりで 1 を入れると「ブレイクポイント」になり、何も消えずに素通りする。エラーは出ない。
- ・気づけたのは点の数を数えていたから。切ったはずなのに合流後の点数が元のちょうど2倍 (3,200) になり、stitch が 1,600本 (=元の点数そのもの) できた。切れていない2枚が丸ごと重なっていたと分かった。「思ったとおりに動いたか」を別の量で確かめる。
- ・面の印は、点ではなくプリミティブに付ける。attribwrangle の実行対象を Primitives にして、面の重心 @P が線より下かどうかで振り分ける。点に付けると、1枚の面の4隅で答えが割れて、切り口がぎざぎざに崩れる。

次にやること

- ・衝突する物にぶつけて破る。今回は重さだけで外している
- ・切る線を画像やカーブから作り、狙った形に破る
- ・033 以降、まだ触っていない分野に進む

布を物にぶつけて破る ― 破れ始めは当たった場所、そのあとは全体へ

2026-09-24 | Houdini 21.0.700 | /obj/impact | ノード 25 / 接続 42

何を作ったか

実験032では布の重さだけで貼り目を外した。破れる線は自分で決めた波で、どこが破れるかは最初から決まっていた。今回はぶつける。格子状に切れ目を入れた布を吊るし、球を投げ込む。

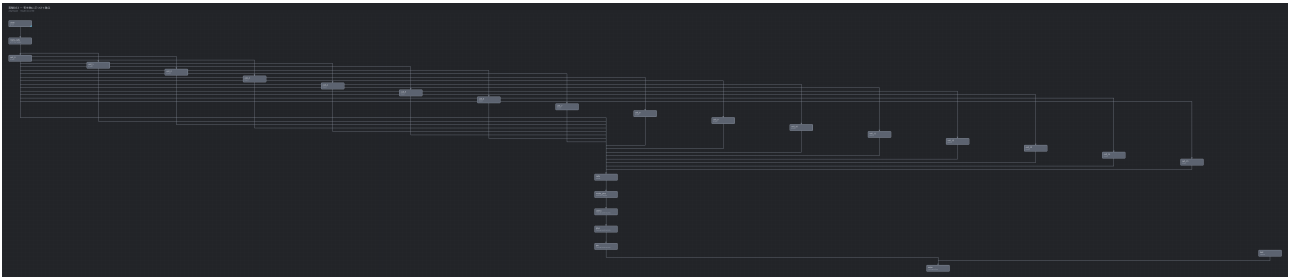
最初の測り方は失敗した。しきい値を 1.0 にしたところ、球のあり／なしで外れた数がまったく同じ 146 本になった。重さだけで全部外れてしまい、球の影響が見えなくなっていたためである。「重さでは外れない」しきい値まで上げて測り直した。

そうすると、しきい値 100～800 では重さだけでは1本も外れず、球を通すと 80／55／55／43／4本 が外れた。球が原因であることは、これで疑いようがない。速さを上げれば外れる数も増える（0.08で0本、0.64で71本）。

ただし「当たった場所に集中する」とは言えなかった。最初に外れる1本は球の通ridorの内側だが、最後には内側 20.0%・外側 21.8%

とほぼ同じ割合になる。破れ始めは局所でも、そのあとは布全体へ広がっていく。

ノードの接続



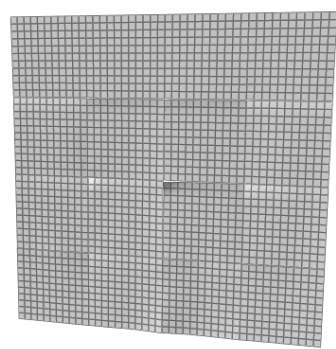
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
cloth	grid	cols = 45 orient = 0 rows = 45 sizex = 3.0 sizey = 3.0 ty = 2.2
mark_cells	attribwrangle	class = 1 snippet = int cx = int(floor((@P.x + 1.5) / 3.0 * 4)); int cy = int(floor((@P.y - 0.7) / 3.0 * 4)); i@cell = clamp(cx, 0, 3) * ...
cell_0	blast	group = @cell!=0 grouptype = 4
cell_1	blast	group = @cell!=1 grouptype = 4
cell_2	blast	group = @cell!=2 grouptype = 4
cell_3	blast	group = @cell!=3 grouptype = 4

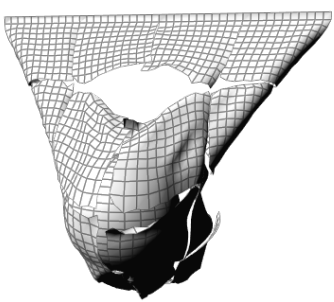
ノード	タイプ	デフォルトから変更したパラメータ
cell_4	blast	group = @cell!=4 grouptype = 4
cell_5	blast	group = @cell!=5 grouptype = 4
cell_6	blast	group = @cell!=6 grouptype = 4
cell_7	blast	group = @cell!=7 grouptype = 4
cell_8	blast	group = @cell!=8 grouptype = 4
cell_9	blast	group = @cell!=9 grouptype = 4
cell_10	blast	group = @cell!=10 grouptype = 4
cell_11	blast	group = @cell!=11 grouptype = 4
cell_12	blast	group = @cell!=12 grouptype = 4
cell_13	blast	group = @cell!=13 grouptype = 4
cell_14	blast	group = @cell!=14 grouptype = 4
cell_15	blast	group = @cell!=15 grouptype = 4
cells	merge	(変更なし)
make_pins	attribwrangle	snippet = if (@P.y > 3.63) { @group_pins = 1; }
cloth1	vellumconstraints	constrainttype = 3
glue	vellumconstraints	breakthreshold = 200.0 constrainttype = 12 dobreaking = 1
pin	vellumconstraints	constrainttype = 6 group = pins grouptype = 1
solve	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618
ball	sphere	radx = 0.42 rady = 0.42 radz = 0.42 rows = 24 tx = 0.0 ty = 2.2 type = 2 tz = -2.6

できたもの

3×3の布を45×45に分割し、4×4のマスキに切り分けて glue で貼り直した。上端だけをピンで留めて吊るし、奥から球（半径0.42）を投げ込む。カメラは2枚とも同じ。



Houdini



Houdini

球なし。しきい値200では重さだけでは1本も外れず、そのまま垂れる。

球あり。同じカメラ・同じフレーム。穴が開いて破片が落ちていく。

調べたこと	結果
元の布	2,025点 / 1,936面
4×4に切ったあと	2,304点 (+279)
作られた拘束	distance 6,160 / bend 5,456 / stitch 261
球	半径 0.42、1フレームあたり z へ 0.16 進む
球が布を通るフレーム	F17 あたり

1. 球を通すと、外れる貼り目は増えるか

しきい値 1.0 では重さだけで全部外れてしまい、球のあり／なしで差が出なかった。上げていくと、重さでは外れず球でだけ外れる範囲が現れる。

しきい値	球なし	球あり	差	最下点
1.0	146	146	0	-7.979
100.0	0	80	+80	1.044
200.0	0	55	+55	1.031
300.0	0	55	+55	1.127
500.0	0	43	+43	1.190
800.0	0	4	+4	1.201
1600.0	0	0	0	1.201

2. 外れた貼り目は、球の通りの近くに偏っているか（しきい値 200）

球の通り道（x=0, y=2.2）から半径 0.9 の内と外に分ける。内側に 105本、外側に 156本の貼り目がある。位置はシミュレーション前の位置で数えた。

フレーム	外れた計	内で外れた	外で外れた	内の率	外の率
14	0	0	0	0.0%	0.0%

フレーム	外れた計	内で外れた	外で外れた	内の率	外の率
18	0	0	0	0.0%	0.0%
22	1	1	0	1.0%	0.0%
28	37	16	21	15.2%	13.5%
36	55	21	34	20.0%	21.8%

3. 球の速さを変えると、外れる数は増えるか（しきい値 500）

1フレームあたりに進む距離を変えた。同じしきい値のまま、速さだけを変えている。

1フレームあたりの距離	外れた貼り目	最下点
0.08	0本	0.712
0.16	43本	1.190
0.32	56本	0.185
0.64	71本	1.434

わかったこと

- ・最初の測り方は失敗した。しきい値 1.0 で測ったところ、球のあり／なしで外れた数がまったく同じ 14 6本だった。重さだけで全部外れていて、球の影響が埋もれていた。差が出ないときは「効かない」ではなく「他の原因で飽和している」ことを先に疑う。
- ・しきい値を上げると、球の効果がはっきり出た。100～800 では重さだけで 0本、球を通すと 80／55／55／43／4本。球なしが 0 なので、外れた分はすべて球のせいである。
- ・速さを上げると、外れる数も増えた。1フレームあたり 0.08 で 0本、0.16 で 43本、0.32 で 56本、0.64 で 71本。単調に増える。ぶつける強さで破れ具合を決められる。
- ・「当たった場所に集中する」とは言えなかった。最後の時点で、内側 20.0%・外側 21.8%。ほぼ同じ割合である。当初の見立ては外れた。
- ・ただし破れ始めは局所だった。最初に外れた1本（F22）は球の通りの道の内側にある。F28 では内 15.2% 対 外 13.5% でわずかに内側が先行し、F36 で追いつかれる。「当たったところから破れ始め、そのあと全体へ広がる」が正しい言い方になる。
- ・広がる理由は測っていないが、布がぶつけられて大きく揺れるので、その動き自体が残りの貼り目に力をかけていると考えられる。ここは確かめていないので断定しない。局所に留めたいなら、球を小さく速くするか、揺れが収まる前にフレームを切る必要がある。
- ・貼り目の見分け方でつまづいた。最初は位置で照合したが、布は動くので、シミュレーション前の位置と後の位置が一致しない。ほぼ全部が「外れた」と数えられ、内 100%・外 96.2% という無意味な結果が出た。つないでいる点の番号で見分けるように直した。番号は動かない。
- ・球は sphere の type を Polygon Mesh にしている。実験030で測ったとおり、Polygon では rows / cols が効かない。

次にやること

- ・球を小さく速くして、破れを当たった場所に留められるか試す
- ・破れた縁を見せる絵にする（いまは確認用の速い描画）
- ・034 以降、まだ触っていない分野に進む

毛を揺らす — 曲がりにくさは「垂れる距離」ではなく「形」に効いていた

2026-09-24 | Houdini 21.0.700 | /obj/hairsim | ノード 9 / 接続 11

何を作ったか

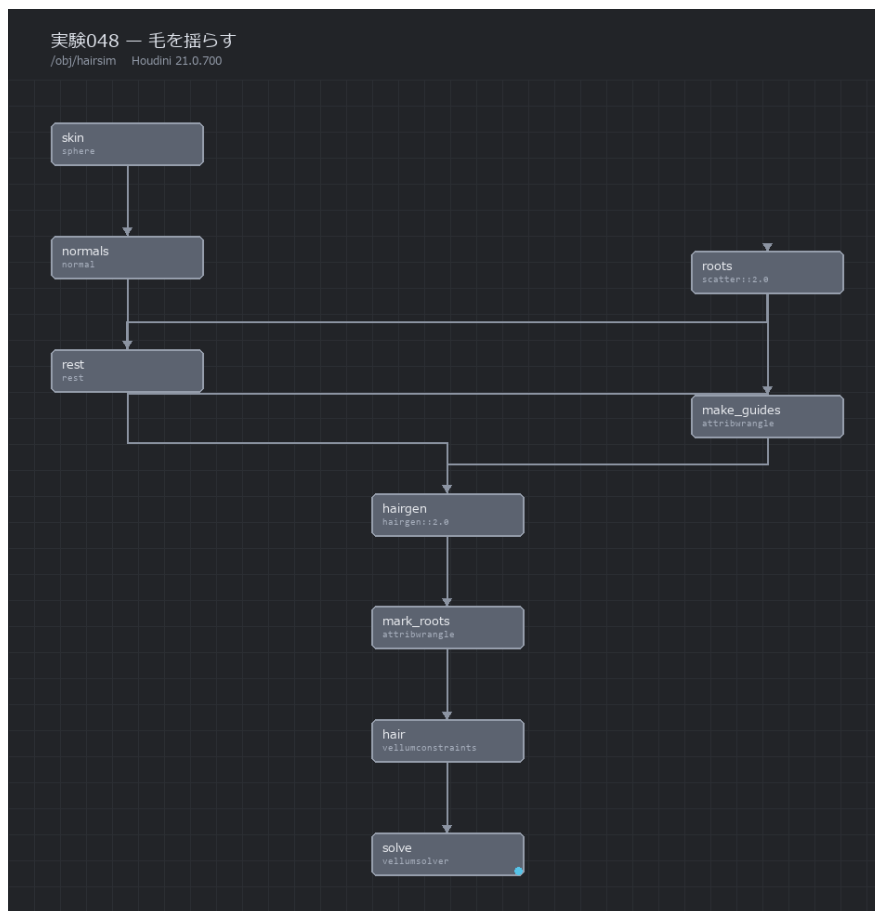
毛はここまで、生やす・束ねる・色を付ける・土台に付いてこさせる、と進めてきた。今回は動かす。vellumconstraints の種類に Hair があり、これでカーブを毛として扱える。

毎回と同じ問いを先に立てる。保たれるはずの量は何か。毛は伸び縮みしないはずなので、毛の長さがそれにあたる。48フレーム流したところ、長さのずれは最大 0.63%。途中のいちばん伸びた瞬間でも 1.49% だった。伸びないのではなく、少し伸びてから戻る。

根元の留め方でつまずいた。vellumconstraints の「Pin to Target」を足しても、拘束は1本も作られず、毛は根元ごと落ちていった。group でも pingroup でも、pintype を3通り変えても同じ。結局、Vellum が見る stopped というアトリビュートを根元の点に直接立てるやり方に変えた。これは効いた。

そして bendstiffness (曲がりにくさ)。0 から 1,000,000 まで100万倍変えても、毛先の高さは -0.5784 から -0.5852 と 1.2% しか変わらない。垂れる距離は決めていない。決めていたのは形のほうだった。毛のまっすぐさ (根元から毛先までの直線距離 ÷ 毛の長さ) は 0.9808 → 0.9994。いちばん曲がった毛で見ると 0.835 → 0.990 とはっきり変わる。100 で頭打ちになる。

ノードの接続

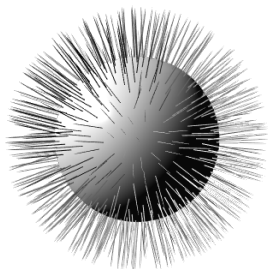


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 30 rows = 30 type = 2
normals	normal	(変更なし)
rest	rest	(変更なし)
roots	scatter::2.0	npts = 120
make_guides	attribwrangle	class = 0 snippet = int segs = 10; float len = 0.6; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = point(0, "P", i); ...
hairgen	hairgen::2.0	density = 50.0 hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0 influenceradius = 0.25 thickness = 0.012
mark_roots	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); setpointgroup(0, "roots", pts[0], 1); setpointattrib(0, "stopped", pts[0], 1);
hair	vellumconstraints	bendstiffness = 0.05 constrainttype = 4 domass = 1
solve	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618

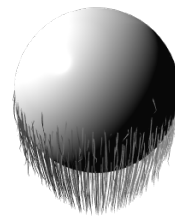
垂れる前と後

毛 620本、長さ 0.596、太さ 0.012。重力だけ。カメラは2枚とも同じ。OpenGL。



Houdini

フレーム1。毛は土台から真っすぐ外へ伸びている。



Houdini

フレーム48。根元は留まったまま、毛先が下へ垂れる。

frame	毛先の高さ 平均	中心からの距離	長さの平均	長さのばらつき	最初との比
1	0.000322	1.590527	0.596209	0.002767	1.000000
4	-0.042383	1.585333	0.597296	0.003049	1.001824
8	-0.241343	1.495563	0.597843	0.003018	1.002741
12	-0.507783	1.244712	0.599988	0.002973	1.006338
18	-0.432239	0.814113	0.598458	0.002978	1.003772
24	-0.446271	0.871080	0.598285	0.002919	1.003481
36	-0.512676	1.288071	0.598500	0.002836	1.003842
48	-0.579763	1.080746	0.599089	0.002810	1.004830

bendstiffness を100万倍まで変える

48フレーム目で測った。まっすぐさは「根元から毛先までの直線距離 ÷ 毛の長さ」。1
なら一直線、小さいほど曲がっている。

bendstiffness	毛先の高さ 平均	中心からの距離	まっすぐさ	いちばん曲がった毛	長さのずれ
0	-0.578420	1.081628	0.980830	0.834974	0.4800%
0.05	-0.579763	1.080746	0.981749	0.811067	0.4830%
1	-0.585164	1.085267	0.990014	0.946751	0.5161%
100	-0.585448	1.098518	0.999359	0.990332	0.5162%
10,000	-0.584058	1.097281	0.999803	0.982212	0.5223%
1,000,000	-0.584029	1.095081	0.999606	0.978315	0.5441%

わかったこと

- ・毛の長さは保たれた。ずれは最大 0.63%。いちばん伸びた瞬間でも 1.49%。伸びないのではなく、少し伸びてから戻る。Vellum の拘束は硬いばねであって、鎖ではない。
- ・根元を留めるのに「Pin to Target」は効かなかった。足しても拘束は1本も作られず、毛は根元ごと落ちていく。group でも pingroup でも、pintype を Permanent / Stopped / Soft と3通り変えても結果は同じだった。Vellum が見る stopped アトリビュートを根元の点に直接立てると効いた。
- ・domass（質量を計算する）は既定でオフ。オフのままだと mass が -1 のまま入る。オンにしてから測っている。
- ・bendstiffness は「どこまで垂れるか」を決めていなかった。0 から 1,000,000 まで100万倍変えても、毛先の高さは -0.5784 → -0.5852 と 1.2% しか動かない。
- ・決めていたのは形のほうだった。まっすぐさは 0.980830 → 0.999359 と上がり、いちばん曲がった毛で見ると 0.834974 → 0.990332。固い毛は「同じところまで垂れるが、まっすぐなまま垂れる」。
- ・100 で頭打ちになる。100 と 10,000 と 1,000,000 でまっすぐさは 0.999359 / 0.999803 / 0.999606。上げ続けても変わらない。これ以上はほぼ一直線なので、変わりようがない。
- ・「平均」だけ見ていたら見落としていた。まっすぐさの平均は 0.9808 → 0.9994 で 1.9% の差にすぎない。いちばん曲がった毛で見ると 0.835 → 0.990 で 18.6%

の差になる。効果が一部にしか出ないときは、平均ではなく端を見る。

・毛先の高さは一本調子に下がらない。-0.508 (12) → -0.432 (18) → -0.446 (24) →
-0.513 (36) →
-0.580 (48)。いったん振り戻している。重力で落ちて、拘束で引き戻されて、揺れている。

次にやること

- ・毛に風を当てる（実験029の popwind と同じ問いを毛で）
- ・サンプル数と毛のザラつきの関係（実験018と同じ問いを毛で）
- ・手順ページをグルームの分だけそろえる

毛に風を当てる ― 実験029と同じ「二次の抵抗」だった

2026-09-24 | Houdini 21.0.700 | /obj/hairwind | ノード 9 / 接続 11

何を作ったか

実験029で、粒に当てる popwind は二次の抵抗 ($dv/dt = a(W-v)^2$) だと突き止めた。毛の場合はどうか。

毛は伸びないので、流される量には上限がある。根元から毛先までの距離は、どんなに強い風でも毛の長さを超えられない。つまり比例するはずがなく、どこかで頭打ちになる。問題はその近づき方で、候補を2つ先に立てた。一次の抵抗なら $s = W / (W + k)$ 、二次なら $s = W^2 / (W^2 + k^2)$ 。s

は「流され具合」= (毛先の x - 根元の x) ÷ 毛の長さ。

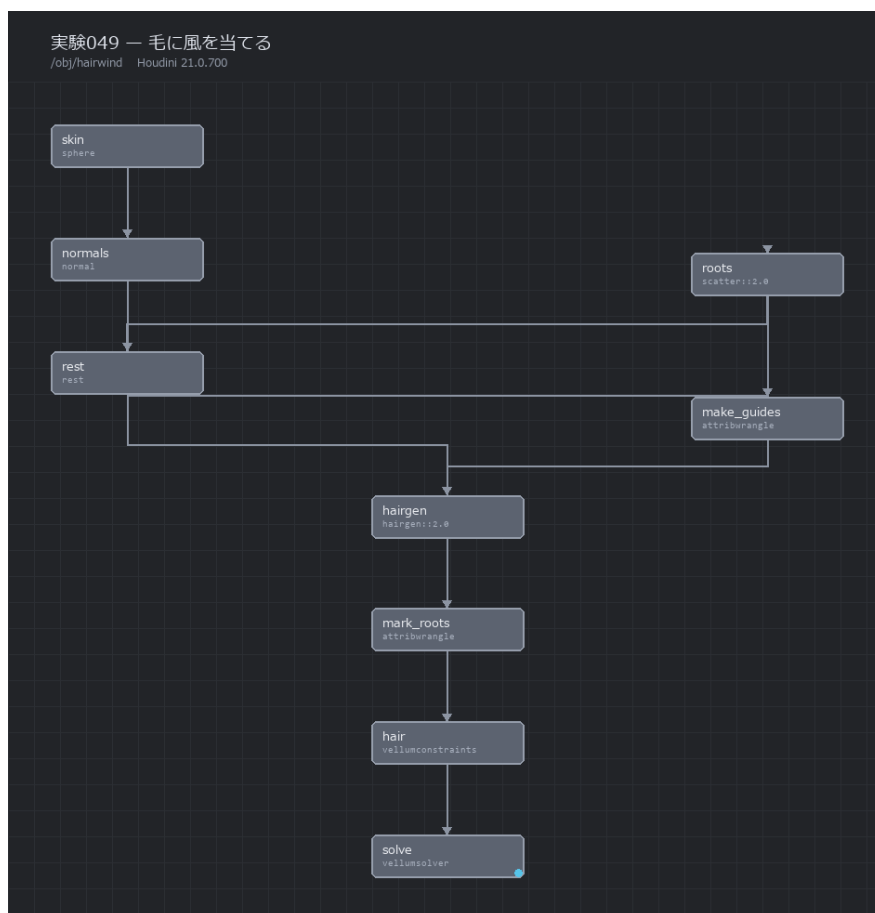
風速を 2 / 5 / 10 / 20 / 40 と変えて測り、k を総当たりで合わせた。残差の二乗和は一次が 0.026267、二次が 0.007404。二次のほうが 3.5倍よく乗る。粒のときと同じだった。

ただしどちらもぴったりにではない。二次でも最大 0.054

ずれる。そして強い風では毛が伸びる。長さは風速 0 で 0.599089 だったものが、40 では 0.631461 と 5.9% 伸びた。重力だけの実験048では 0.63% しか伸びなかったので、これは風の分である。Vellum の拘束は硬いばねであって鎖ではない、という実験048の結論が、強い力をかけると目に見える形で出てくる。

。

ノードの接続

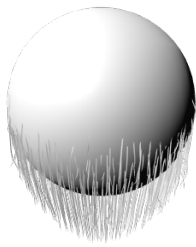


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 30 rows = 30 type = 2
normals	normal	(変更なし)
rest	rest	(変更なし)
roots	scatter::2.0	npts = 120
make_guides	attribwrangle	class = 0 snippet = int segs = 10; float len = 0.6; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = point(0, "P", i); ...
hairgen	hairgen::2.0	density = 50.0 hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0 influenceradius = 0.25 thickness = 0.012
mark_roots	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); setpointgroup(0, "roots", pts[0], 1); setpointattrib(0, "stopped", pts[0], 1);
hair	vellumconstraints	bendstiffness = 0.05 constrainttype = 4 domass = 1
solve	vellumsolver	windshadow_maskfalloff1pos = 0.8500000238418579 windshadow_maskfalloff1value = 1.0 windshadow_maskfalloff2pos = 0.9973856210708618 windspeed = 20.0 windx = 1.0

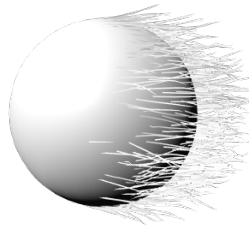
風のあるなし

毛 620本、長さ 0.596、太さ 0.012。風は +x 向き。カメラは2枚とも同じ。48フレーム目。OpenGL。



Houdini

風速 0。重力だけで下に垂れている。



Houdini

風速 40。毛が風下へ寝て、流され具合は 0.950279。

風速	流され具合	ばらつき	毛先の x	毛先の高さ	長さの平均
0	-0.004210	0.117582	0.014733	-0.579763	0.599089
2	0.318497	0.062716	0.207969	-0.552739	0.598813
5	0.736690	0.081876	0.457701	-0.314031	0.597901
10	0.868605	0.073819	0.537896	-0.139250	0.599409
20	0.923036	0.045864	0.578377	-0.042630	0.607896
40	0.950279	0.028984	0.617211	-0.023428	0.631461

2つの式にあてはめる

風速 0 のときの値 (-0.004210) を 0 とみなして正規化し、k を 0.01 刻みで総当たりして残差の二乗和がいちばん小さい値を選んだ。

風速	実測	一次 $W/(W+k)$	二次 $W^2/(W^2+k^2)$
2	0.321354	0.445434	0.303469
5	0.737794	0.667557	0.731403
10	0.869156	0.800641	0.915911
20	0.923358	0.889284	0.977563
40	0.950487	0.941398	0.994295

どちらの式が合うか

残差の二乗和が小さいほどよく乗る。

式	k	残差の二乗和	1点あたりの平均のずれ	最大のずれ
一次 $W/(W+k)$	2.49	0.026267	0.0725	0.1241
二次 $W^2/(W^2+k^2)$	3.03	0.007404	0.0385	0.0542

わかったこと

- ・二次の抵抗のほうがよく乗る。残差の二乗和が一次 0.026267 に対して二次 0.007404 で、3.5倍の差。実験029で粒について突き止めたのと同じ形だった。粒でも毛でも、Houdini の風は速度の差の二乗で効く。
- ・低い風速でいちばんはっきり分かれる。風速2 での実測は 0.321354。一次の式は 0.445434 と大きく外し、二次の式は 0.303469 と近い。頭打ちに入ったあと（10以上）はどちらも似た値になるので、区別は弱い風のところでつく。
- ・それでもぴったりではない。二次でも最大 0.054 ずれる。この式で完全に説明できるとは言えない。毛は粒と違って根元が固定されており、隣の毛ともぶつかる。
- ・強い風では毛が伸びる。長さは風速 0 で 0.599089、40 では 0.631461 と 5.9% 伸びた。重力だけの実験048では 0.63% だったので、これは風の分。Vellum の拘束は硬いばねであって鎖ではない、というのが強い力で見えてくる。
- ・風は毛をそろえる。流され具合のばらつきは 0.117582（風速0）→ 0.028984（風速40）と 4分の1になる。強い風の下では、どの毛もほぼ同じ向きになる。
- ・毛先は上がってくる。高さの平均は -0.579763（風速0）→ -0.023428（風速40）。横向きの風なのに縦の位置が変わるのは、毛が水平に近づくからで、重力に逆らって持ち上がっているわけではない。
- ・風速 0 でも流され具合は 0 ではない（-0.004210）。球の上に散らした毛なので、風下側・風上側が均等に混ざっていて、平均するとほぼ 0 になる。この値を差し引いてから式にあてはめている。
- ・測り方について。流される量そのものではなく「毛の長さで割った値」を見ている。こうすると 0~1 に収まり、上限が 1 だとはっきりする。上限のある量は、上限で割ってから比べる。

次にやること

- ・毛どうしのぶつかり（衝突）を入れると式がどう変わるか
- ・サンプル数と毛のザラつきの関係（実験018と同じ問いを毛で）
- ・実験050の節目 — グループ編のまとめ

旗を風になびかせる — 風速 2

まではほぼ垂れたまま。持ち上がり方は「抵抗 × 風速²」でそろろう

2026-09-24

何を作ったか

左端を竿に留めた布（Vellum の cloth）に、ソルバの Built-in Wind で横から風を当てた。旗の角度は、竿の真ん中から見た「旗の先の辺」の向きで測る。0度が水平、90度が真下に垂れた状態。

最初は3秒回して最後のフレームで角度を取ったが、旗はまだ大きく揺れていて、1フレームの値は ±8～16度もぶれた。風速 2 で抵抗を 20 から 40 に上げると角度が増える、という逆向きの結果まで出た。そこで6秒回し、最後の2秒（48フレーム）の平均で測り直した。

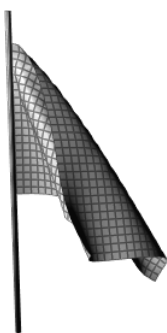
風速 0 / 1 / 2 では、旗はほとんど持ち上がらない（85 / 85 / 78度）。4 で 41度、8 で 20度、16 で 9度と、そこから急に水平へ近づく。強い風ほど、はためきも大きい。風速 16 では角度が -6～22度の間で揺れ続けた。

布の Normal Drag（面に垂直な向きの抵抗、既定 10）も同じ向きに効く。風速 4 のまま 1 / 10 / 100 にすると、89 / 41 / 17度。

では、効いているのは「抵抗 × 風速」か「抵抗 × 風速²」か。積をそろえた組を並べた。抵抗 × 風速² が 160 の組は、（風速 4 ・ 抵抗 10）41.1度、（風速 2 ・ 抵抗 40）40.5度とほぼ一致した。抵抗 × 風速 が 40 の組は、41.1 / 78.5 / 30.6度とばらばら。風の効きは速さの2乗に近い。実験029（POP の風）・079（popdrag）と同じ形だった。ただし（風速 8 ・ 抵抗 2.5）だけは 50.5度で、9度ずれた。

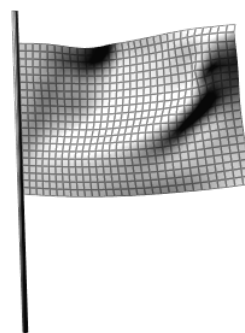
完成した旗（6秒後）

3 × 2 の布（31 × 21 点）を、左端の点だけ竿に留めた。Normal Drag 10。2枚とも同じカメラ。



Houdini

風速 4。斜め下へ垂れながら、先がはためく（平均 41度）。

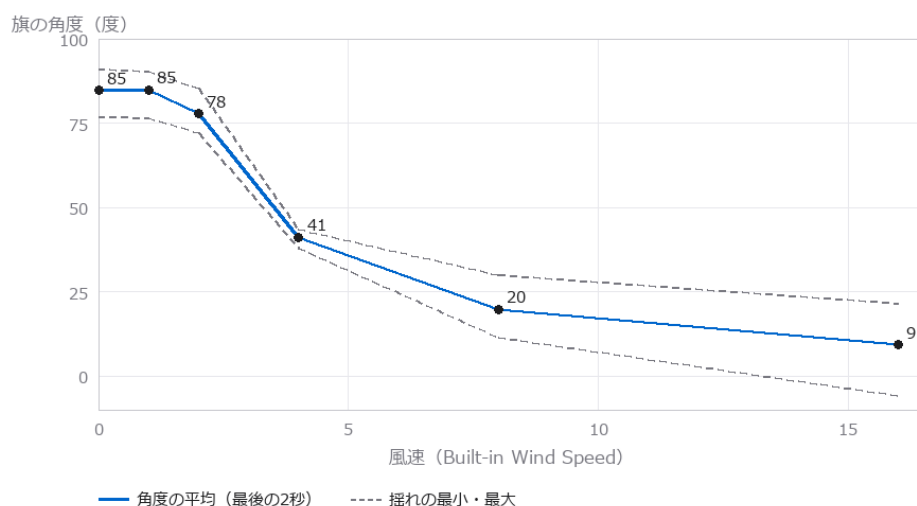


Houdini

風速 16。ほぼ水平に広がり、面が大きく波打つ（平均 9度、-6～22度で揺れる）。

A. 風速を変える（Normal Drag 10 ・ 最後の2秒の平均）

点線は、最後の2秒の間の角度の最小と最大。



風速 2 までは垂れたまま。4 から急に持ち上がり、そのあとはゆっくり水平へ近づく。

風速	角度の平均	揺れ (標準偏差)	最小～最大	計算の秒 (6秒ぶん)
0	84.69	4.83	77.0～91.1	4.56
1	84.77	4.45	76.6～90.4	4.52
2	77.72	4.67	72.1～85.3	4.48
4	41.08	1.82	38.0～43.5	4.21
8	19.72	5.59	11.5～30.1	3.55
16	9.31	8.72	-5.8～21.6	3.64

B・C. 抵抗と風速の組み合わせ

「抵抗 × 風速²」が同じ 160 の3組のうち、2組は 0.6度差でそろった。「抵抗 × 風速」が同じ 40 の3組は、31～79度とばらばら。

風速	Normal Drag	抵抗 × 風速	抵抗 × 風速 ²	角度の平均	揺れ
4	1	4	16	88.81	4.88
4	10	40	160	41.08	1.82
4	100	400	1600	16.62	3.11
2	40	80	160	40.51	3.07
8	2.5	20	160	50.45	2.75
2	20	40	80	78.51	7.92
8	5	40	320	30.59	7.87

わかったこと

- ・風速 2 までは、旗はほとんど持ち上がらない。0 / 1 / 2 で 85 / 85 / 78度。4 で 41度、8 で 20度、16 で 9度。「なびかせる」なら 4 以上から。
- ・風が強いほど、はためきも大きい。最後の2秒の角度の揺れは、風速 4 で ±1.8度、16 で ±8.7度（-6～22度）。面も大きく波打つ。
- ・布の Normal Drag も同じ向きに効く。風速 4 で 1 / 10 / 100 にすると 89 / 41 / 17度。抵抗を上げて風を強くしても、旗は持ち上がる。

- ・持ち上がり方は「抵抗 × 風速²」でほぼそう。積が 160 の (4, 10) と (2, 40) は 41.1度と 40.5度。「抵抗 × 風速」がそろった組は 31~79度とばらばら。風の効きは速さの2乗に近く、POPの風（実験029）・空気抵抗（実験079）・毛（実験049）と同じ形だった。

- ・ただし完全にはそろわない。(風速 8・抵抗 2.5) は同じ積 160 でも 50.5度で、9度ずれた。抵抗がとても小さいときに外れるのかは、確かめていない。

- ・旗は3秒ではまだ落ち着かない。3秒後の1フレームで測ると ±8~16度ぶれ、抵抗を上げたのに角度が増えるという逆向きの値まで出た。6秒回して最後の2秒を平均すると、筋の通った値になった。揺れるものは、1フレームではなく一定時間の平均で測る。

- ・計算の時間は、651点の布で6秒（144フレーム）が 3.6~4.6秒。風が強いほど少し速かった。

次にやること

- ・道に沿って物を並べる
- ・旗の風に乱れ（ノイズ）を入れると、はためき方はどう変わるか
- ・布の細かさ（点の数）で、はためき方は変わるか

vellumconstraints の Cloth は網を三角形にしてから、辺ごとに伸びの拘束、内側の辺ごとに曲げの拘束を作る

2026-09-24

何を作ったか

rows × cols の grid（四角形の網）と、先に divide で三角形にした網に、vellumconstraints（Constraint Type = Cloth）をかけ、2つ目の出力の拘束を type 属性ごとに数えた。

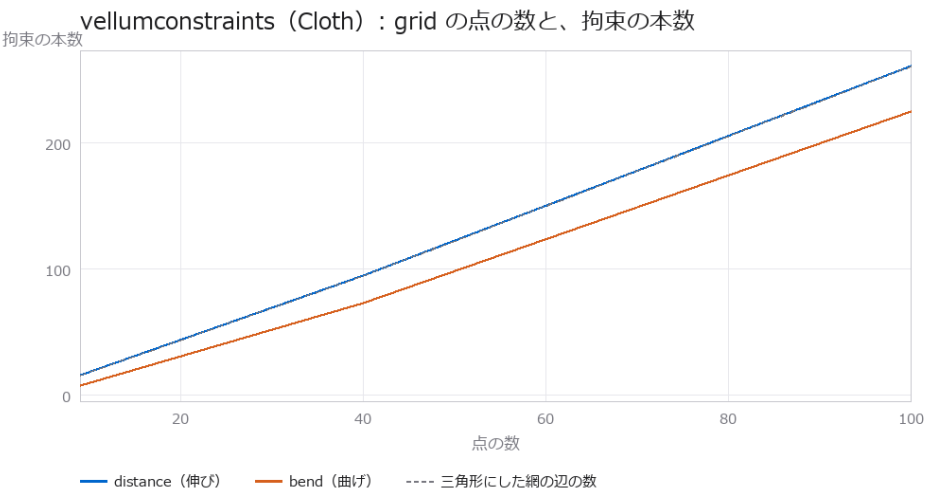
伸び（distance）の拘束は、三角形にした網の辺の数と3通りとも一致。四角形のまま渡しても、四角形の対角線にも拘束が張られる（10×10 で辺 180 本に対して 261 本 = 辺 + 面の数）。

曲げ（bend）の拘束は、三角形にした網の内側の辺の数と一致。2枚の三角形にはさまれた辺ごとに1本で、縁の辺には付かない。

四角形で渡しても、三角形で渡しても、拘束の数は同じ（3通りとも同じ）。Cloth は中で三角形に分けてから拘束を作っている。

拘束づくりは 1 回 0.16 秒前後。網の大きさ（9～100 点）にはほとんど左右されない（最初の1回だけ 0.28 秒）。

網と拘束の本数



伸び（青）は点線（三角形の網の辺の数）に重なる。

grid	網	点	面	辺	内側の辺	distance	bend	合計	秒
3×3	quads	9	4	12	4	16	8	24	0.2774
3×3	triangles	9	8	16	8	16	8	24	0.1619
5×8	quads	40	28	67	45	95	73	168	0.1616
5×8	triangles	40	56	95	73	95	73	168	0.1608
10×10	quads	100	81	180	144	261	225	486	0.1669
10×10	triangles	100	162	261	225	261	225	486	0.1628

わかったこと

- ・ 伸びの拘束 = 三角形の辺、曲げの拘束 = 内側の辺。本数は計算で出せる。
- ・ 四角形の網でも対角線に伸びの拘束が張られる。対角線の向きは divide と同じとは限らない（確かめていない）。
- ・ 点を倍にすると拘束もほぼ倍。計算の重さの見積もりに使える。

次にやること

- ・ Hair（線）のときの拘束の本数
- ・ Stretch Stiffness を変えたときの伸び方

Vellum の布は既定の硬さでも Substeps 1 で 2% 伸びる — Substeps 1 では 10^4 と 10^{10} の差が出ない

2026-09-24

何を作ったか

1×1 の布 (21×21 点) を縦に立て、上の辺を Pin Points で留めて、重力で 2 秒 (49 フレーム) ぶら下げた。vellumconstraints (Cloth) の Stretch Stiffness を 1×10^e で変え、四角形の辺の長さが元の何倍になったかを測った。

****既定 (10^{10}) でも、Substeps 1 ではいちばん伸びた辺が 2.3% 伸びた**** (平均 0.61%、下の辺は 0.0127 下がった)。Substeps を 5 にすると 0.10% まで縮んだ。

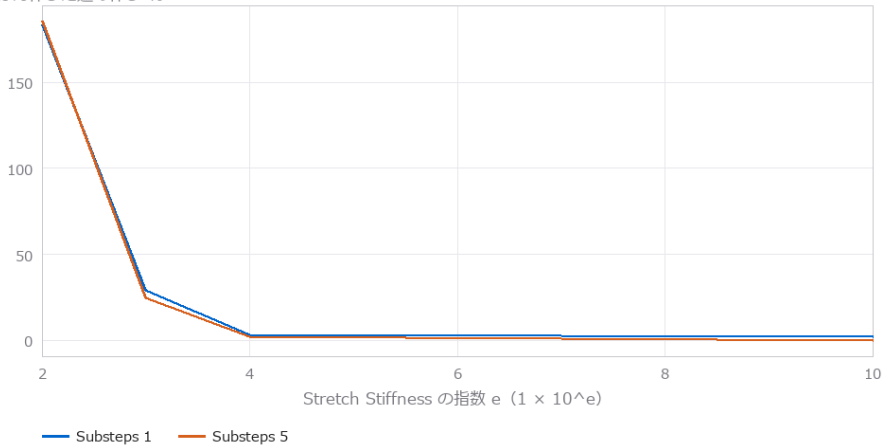
**** 10^4 と 10^{10} は、Substeps 1 ではほぼ同じ伸び**** (3.3% と 2.3%)。硬さをいくら上げて、1 ステップで直しきれる量に上限があり、そこで止まっている。上限を上げるのは Substeps。

**** 10^3 から下では、硬さそのものが効く。* 10^3 で 29%、 10^2 で 184% (布の下の辺が 0.79 下がり、ゴムのよう伸びる)。ここでは Substeps を上げて、伸びはあまり変わらない (10^3 で 29% $\rightarrow$ 25%、 10^2 で 184% $\rightarrow$ 185%)。**

****Substeps 5 の時間は約 2.5 倍**** (49 フレームで平均 1.8 秒 $\rightarrow$ 4.4 秒)。

Stretch Stiffness・Substeps と伸び

Vellum (上の辺を留めた 1×1 の布・2 秒後) : いちばん伸びた辺の伸び率と Stretch Stiffness の指数 e (1×10^e)



Substeps 1 (青) は 10^4 より上で平ら。Substeps 5 (橙) は 10^{10} まで下がり続ける。

Stretch Stiffness	Substeps	いちばん伸びた辺	辺の平均	下の辺の下がり	49 フレームの秒
1e10	1	2.32%	0.61%	0.0127	1.97
1e10	5	0.10%	0.03%	0.0006	4.30
1e4	1	3.27%	0.87%	0.0180	1.64
1e4	5	2.29%	0.51%	0.0103	4.31
1e3	1	29.13%	6.85%	0.1359	1.68
1e3	5	24.78%	5.96%	0.1157	4.51

Stretch Stiffness	Substeps	いちばん伸びた辺	辺の平均	下の辺の下がり	49 フレームの秒
1e2	1	183.71%	40.14%	0.7899	1.73
1e2	5	185.47%	41.55%	0.8399	4.64

わかったこと

- ・ 布が伸びて見えるなら、まず Substeps を上げる。既定の硬さのままで、伸びは 2.3% → 0.1%。
- ・ Substeps 1 なら、Stretch Stiffness は 10^4 より上ではほぼ同じ。Substeps 5 では 10^4 でも 2.3% 伸びるので、硬さも要る。
- ・ Substeps 5 で時間は約 2.5 倍。

次にやること

- ・ Constraint Iterations を増やしたときの伸び
- ・ 布の細かさ（点の数）で伸びがどう変わるか

Vellum の布の伸びは、Substeps より Constraint Iterations を上げる方が安く減る — 400 回で 0.2%、1600 回で 0.002%

2026-09-24

何を作ったか

実験170 の続き。上の辺を留めた 1×1 の布 (21×21 点、Stretch Stiffness は既定の 10^{10}) を 2 秒ぶら下げ、vellumsolver の Constraint Iterations (既定 100) と Substeps を変えて、いちばん伸びた辺の伸びと時間を測った。

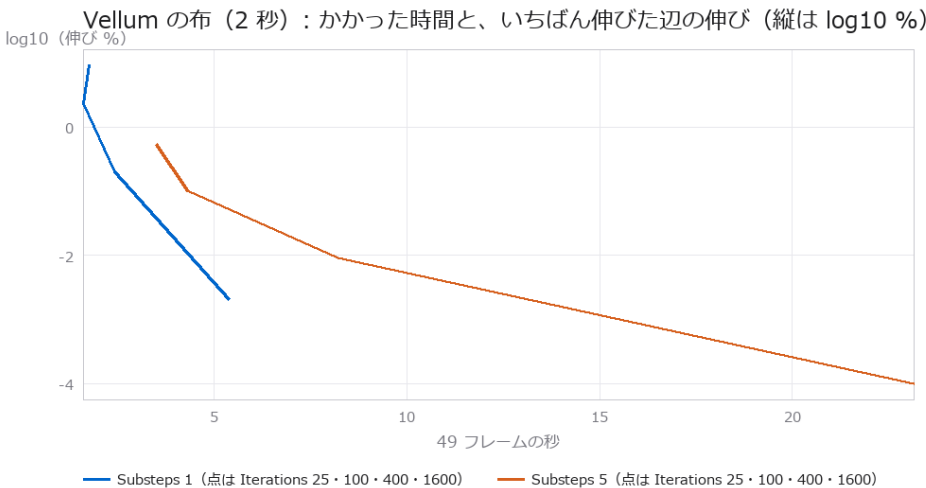
**Iterations を増やすほど、伸びは速く減った。 **Substeps 1 で、25・100・400・1600 回のとき 9.15%・2.32%・0.202%・0.002%。4 倍にするたびに、伸びは 1/4 より大きく減っている。

**同じ時間なら、Iterations の方が伸びが小さい。 **Iterations 400・Substeps 1 は 2.4 秒で 0.20%。既定の Iterations 100 で Substeps 5 にすると 4.3 秒で 0.10%。Iterations 1600・Substeps 1 は 5.4 秒で 0.002% (ほぼ伸びない) 。

**Substeps は、動きの速い物 (ぶつかる・跳ねる) のためにとっておく。 **伸びを抑えるだけなら Iterations で足りる。(この布はゆっくりぶら下がるだけ。速く動く布では、確かめていない。)

Iterations を 4 倍にしても、時間は 1.5~2.8 倍 (決まった手間があるため。Substeps 5・1600 回で 23.1 秒) 。

Iterations・Substeps と、伸び・時間



青 (Substeps 1) の方が、同じ秒数で下 (伸びが小さい) に来る。

Constraint Iterations	Substeps	いちばん伸びた辺	辺の平均	下の辺の下がり	49 フレームの秒
25	1	9.153%	2.255%	0.04680	1.77
25	5	0.526%	0.129%	0.00267	3.52
100	1	2.322%	0.608%	0.01272	1.61
100	5	0.101%	0.027%	0.00055	4.32
400	1	0.202%	0.052%	0.00110	2.42

Constraint Iterations	Substeps	いちばん伸びた辺	辺の平均	下の辺の下がり	49 フレームの秒
400	5	0.009%	0.002%	0.00005	8.23
1600	1	0.002%	0.000%	0.00001	5.41
1600	5	0.000%	0.000%	0.00000	23.14

わかったこと

- ・布が伸びるなら、まず Constraint Iterations を上げる。400 回で 0.2%、時間は 1.5 倍。
- ・Substeps 5 は同じくらいの伸びで時間が約 2.7 倍。
- ・既定（100 回・Substeps 1）では 2.3% 伸びる。ぶら下がる布では見て分かる差。

次にやること

- ・速く動く布（風である）で Iterations と Substeps を比べる
- ・点の数を増やしたときに必要な Iterations